Università degli Studi di Padova

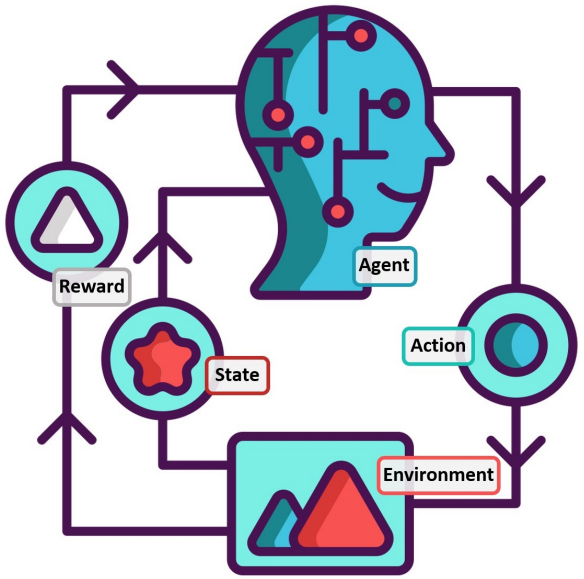
Reinforcement Learning
2024/2025

RL2 Reinforcement Learning Research Lab @ UniPD

Lecture #10
Temporal Difference Learning & n-step Bootstrapping

Gian Antonio Susto

Ruggero Carli



Announcements before start

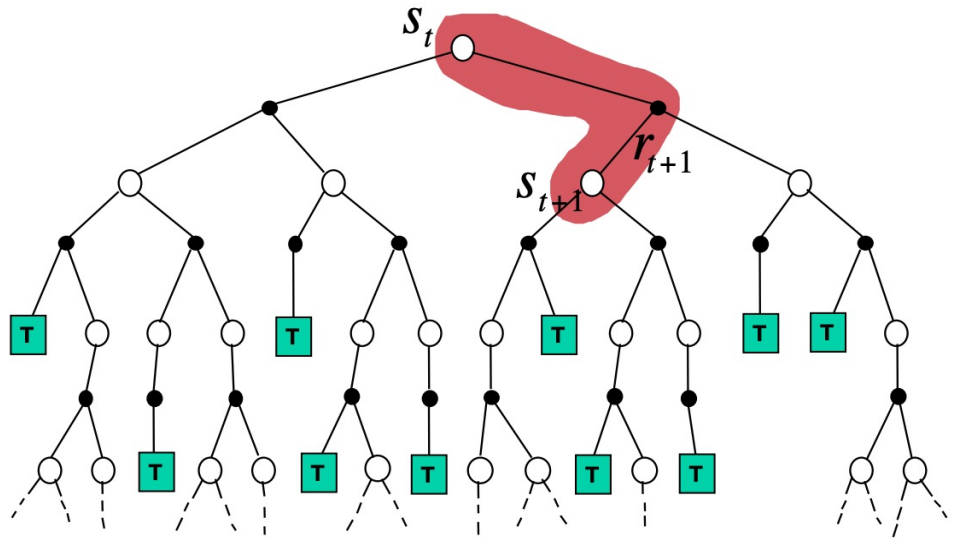
- Starting Thursday, Prof. Carli will be the lecturer (I’ll rejoin the course later).
- The first partial exam will cover content up to today's lecture.
- To finish by the end of 2024, a pre-recorded lecture will be provided on Moodle: the lecture will be on ‘Deep Learning Introduction’ and it will cover basic concepts on Neural Networks (so it may be optional for some of you). Deep Learning will be useful (but not necessary) tool for the project part of the course.

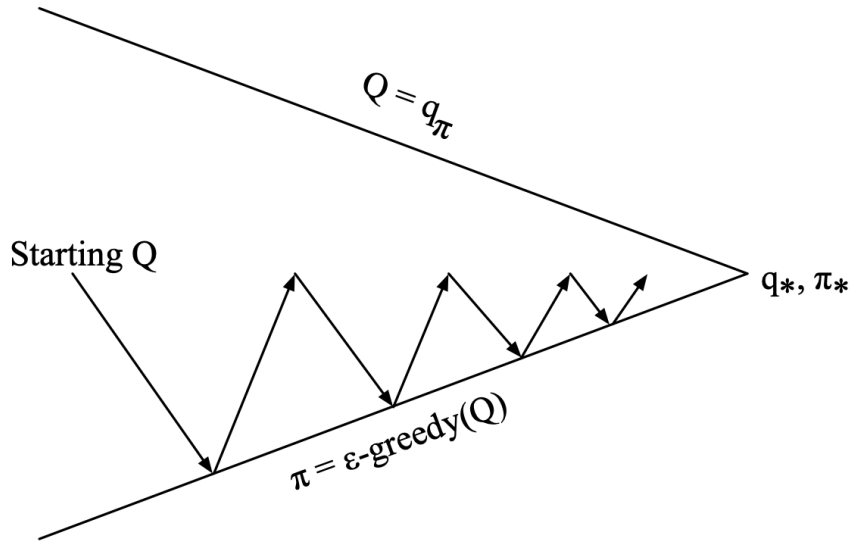
Recap – TD-Learning Control: SARSA & Q-learning

- We have seen how Generalized Policy Iteration could be applied also to the TD-Learning framework
- We have seen 2 approaches

1. SARSA (on-policy)
2. Q-Learning (off-policy)

$$V(S_t) \leftarrow V(S_t) + \alpha\left(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right)$$

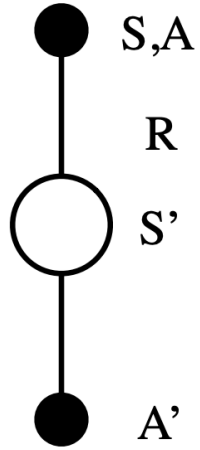




Recap – TD-Learning Control: Q-learning vs. SARSA

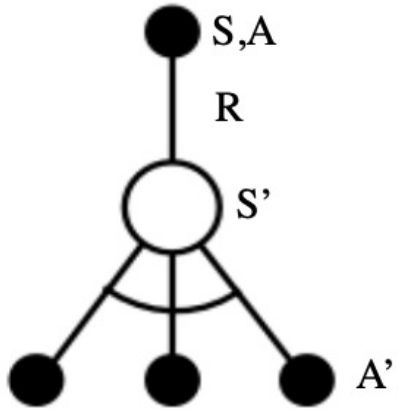
SARSA

Take action A, observe R, S'
Choose A' from S' using policy derived from Q (e.g., ε-greedy)
$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma Q(S',A') - Q(S,A)\right]$
$S \leftarrow S'$; $A \leftarrow A'$;



Q-learning

Choose A from S using policy derived from Q (e.g., ε-greedy)
Take action A, observe R, S'
$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S',a) - Q(S,A)\right]$
$S \leftarrow S'$

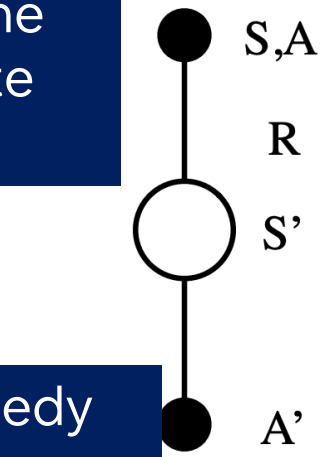


Recap – TD-Learning Control: Q-learning vs. SARSA

SARSA

Take action A, observe R, S'
Choose A' from S' using policy derived from Q (e.g., ε-greedy)
$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma Q(S',A') - Q(S,A)\right]$
$S \leftarrow S'$; $A \leftarrow A'$;

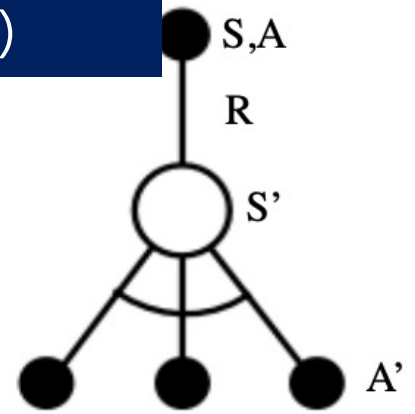
You actual perform next action, according to the policy and then update Q(s,a)



You look ahead and imagine greedy next action to update Q(s,a) (but you then perform the actual next action based on your current policy)

Q-learning

Choose A from S using policy derived from Q (e.g., ε-greedy)
Take action A, observe R, S'
$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S',a) - Q(S,A)\right]$
$S \leftarrow S'$

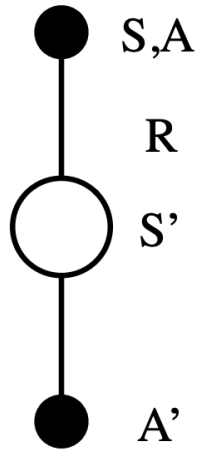


Recap – TD-Learning Control: Q-learning vs. SARSA

SARSA

ON POLICY: We only have one (target) policy here

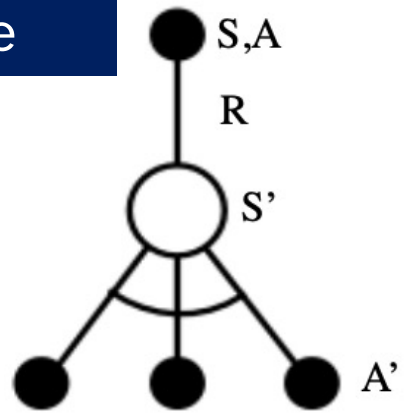
Take action A, observe R, S'
Choose A' from S' using policy derived from Q (e.g., ε-greedy)
$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma Q(S',A') - Q(S,A)\right]$
$S \leftarrow S'$; $A \leftarrow A'$;



Q-learning

OFF-POLICY: We have a behaviour (epsilon-greedy) and a target policy (greedy!) here

Choose A from S using policy derived from Q (e.g., ε-greedy)
Take action A, observe R, S'
$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S',a) - Q(S,A)\right]$
$S \leftarrow S'$



Recap – TD-Learning Control: Q-learning vs. SARSA

Sarsa: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big)$

Bellman Expectation Equation

$$q_\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)\left(r + \gamma \sum_{a'} \pi(a' \mid s') q_\pi(s', a')\right)$$

SARSA is a sample-based version of Policy Iteration

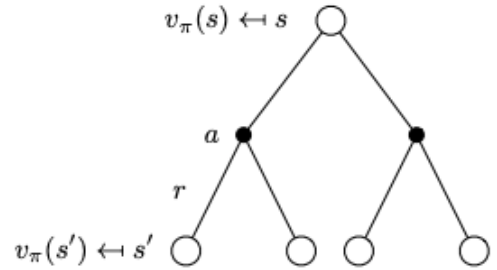
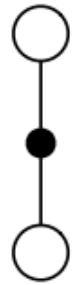
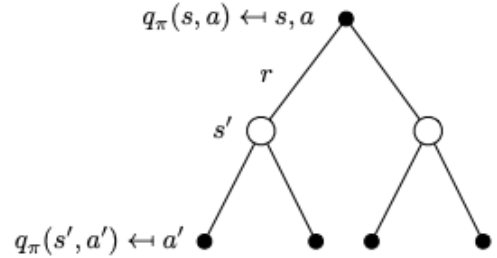
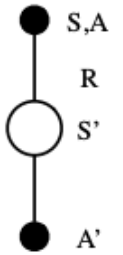
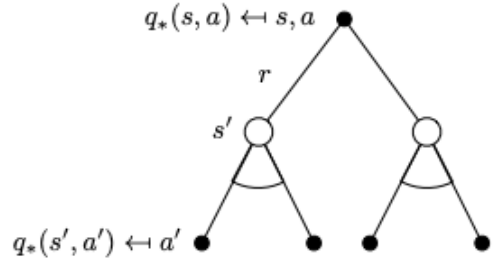
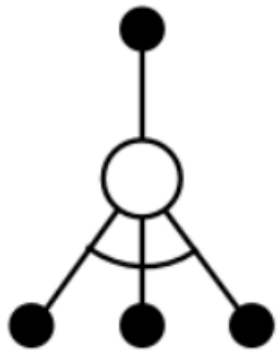
Q-learning: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\big)$

Bellman Optimality Equation

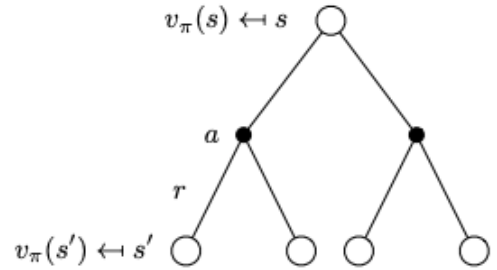
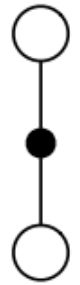
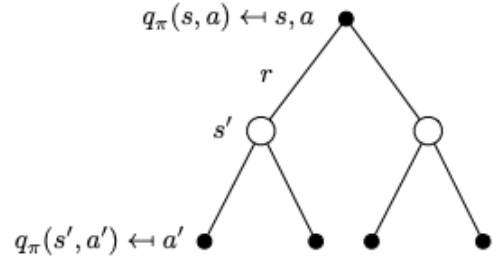
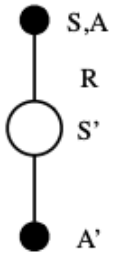
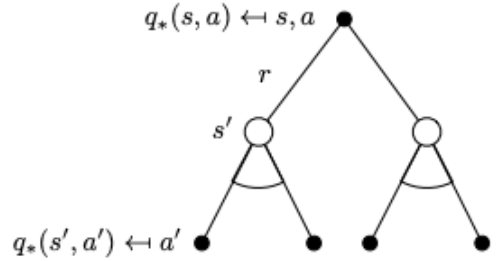
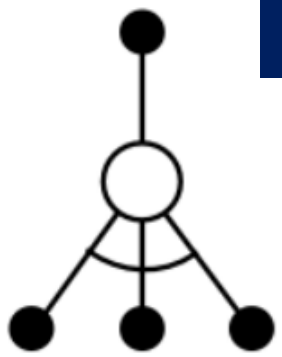
$$q_*(s,a) = \sum_{s',r} p(s', r \mid s, a)\big(r + \gamma \max_{a'} q_\pi(s', a')\big)$$

Q-learning is a sample-based version of Value Iteration

A. White, M. White 'Sample-based Learning Methods'

	Full Backup (DP)	*Sample Backup (TD)*
Bellman Expectation Equation for $v_\pi(s)$	 Iterative Policy Evaluation	 TD Learning
Bellman Expectation Equation for $q_\pi(s,a)$	 Q-Policy Iteration	 Sarsa
Bellman Optimality Equation for $q_*(s,a)$	 Q-Value Iteration	 Q-Learning

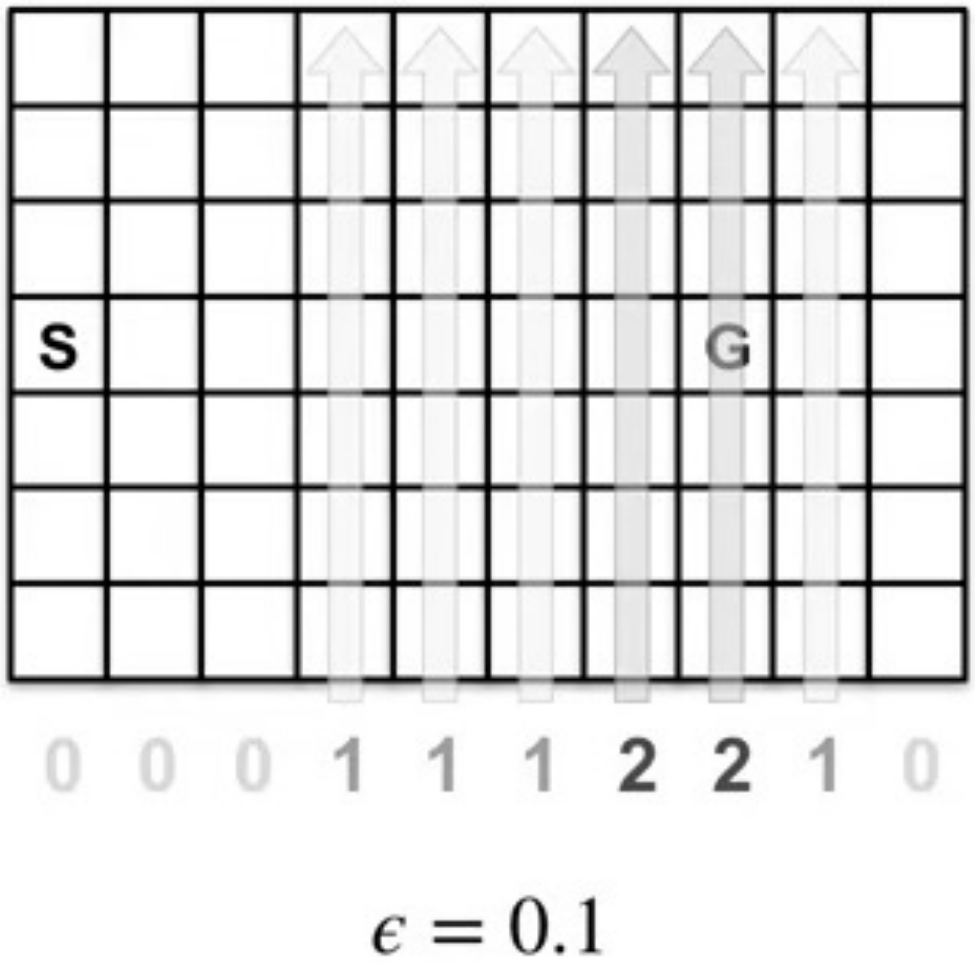
D. Silver 'Reinforcement Learning' @ UCL

	Full Backup (DP)	*Sample Backup (TD)*
Bellman Expectation Equation for $v_\pi(s)$	 Iterative Policy Evaluation	 TD Learning
Bellman Expectation Equation for $q_\pi(s,a)$	 Q-Policy Iteration	 Sarsa
Bellman Optimality Equation for $q_*(s,a)$	 Q-Value Iteration	 Q-Learning

Q-learning is usually faster than SARSA

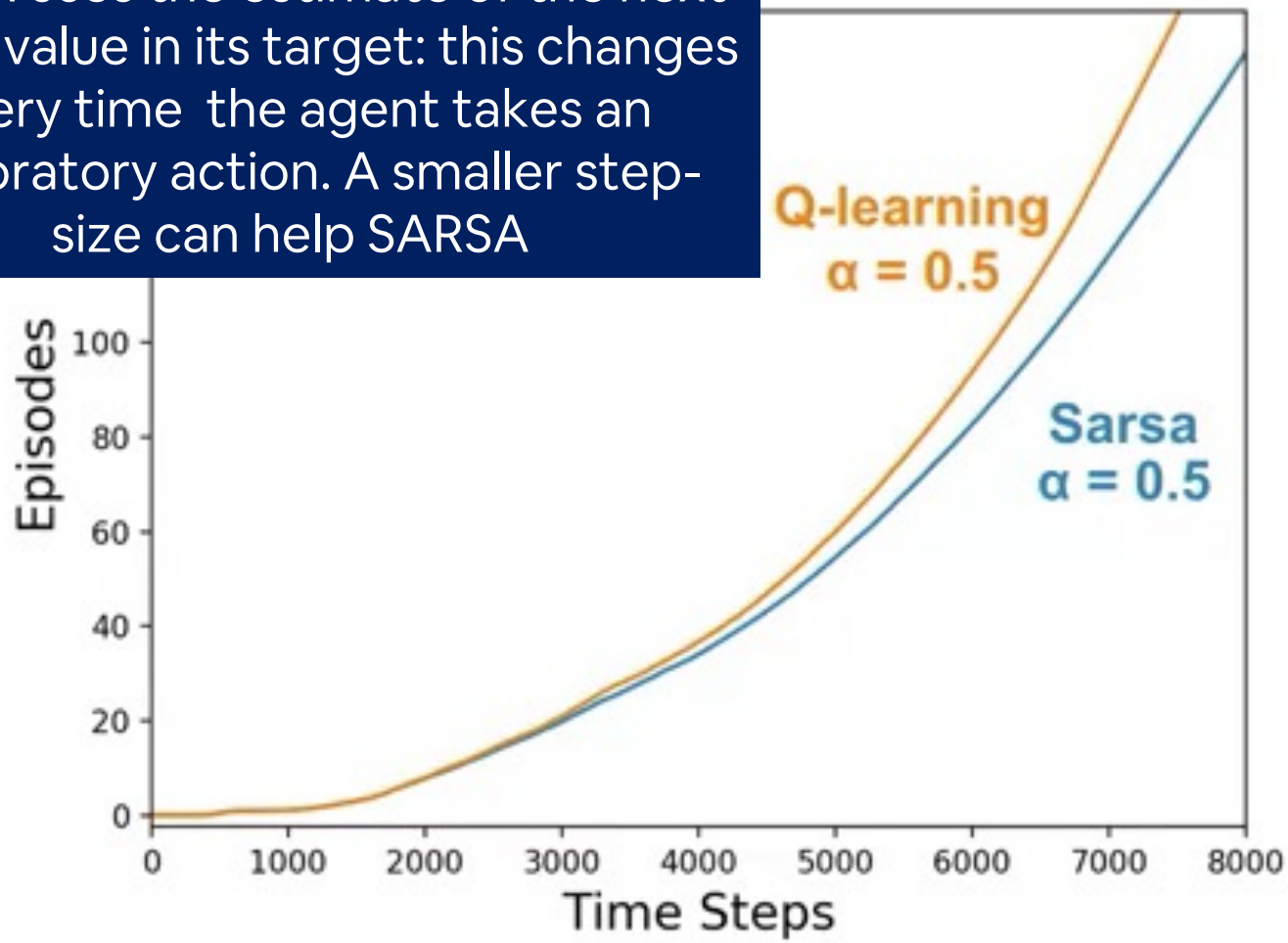
D. Silver 'Reinforcement Learning' @ UCL

Recap - Control: SARSA vs Q-learning – ‘Windy Grid World’ Example #01



$\epsilon = 0.1$

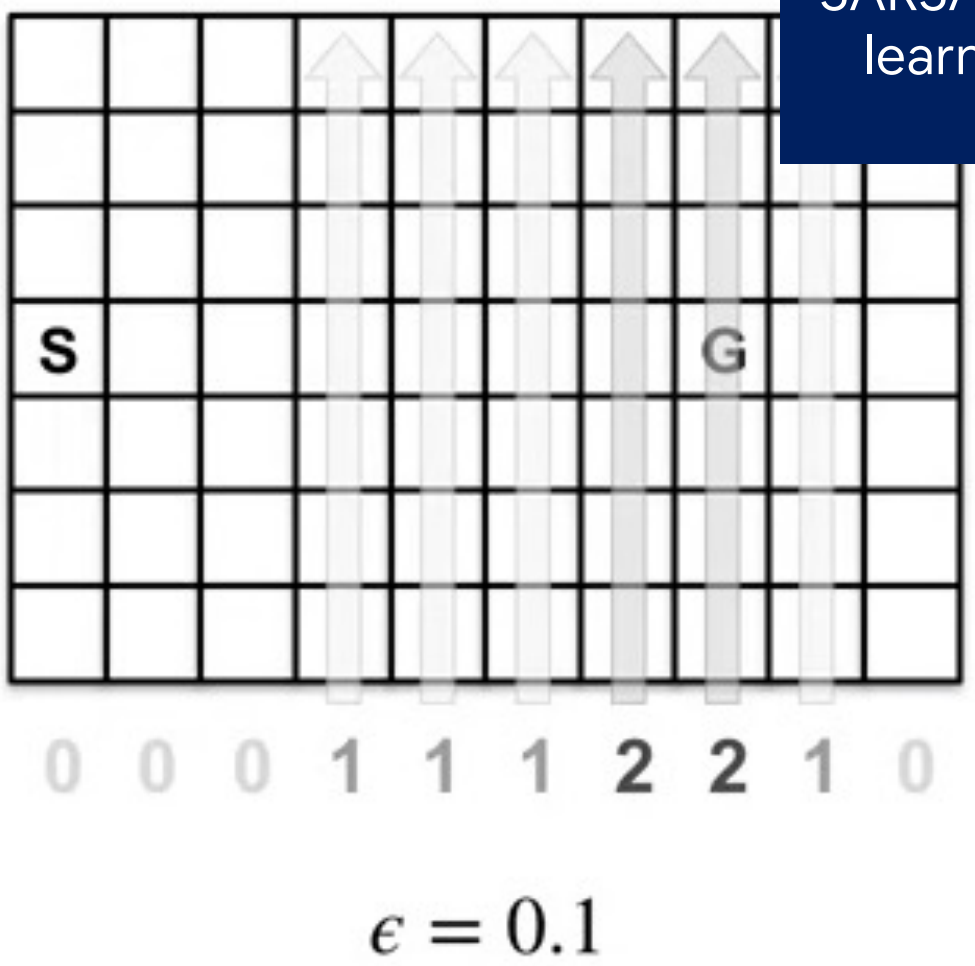
SARSA uses the estimate of the next action value in its target: this changes every time the agent takes an exploratory action. A smaller step-size can help SARSA

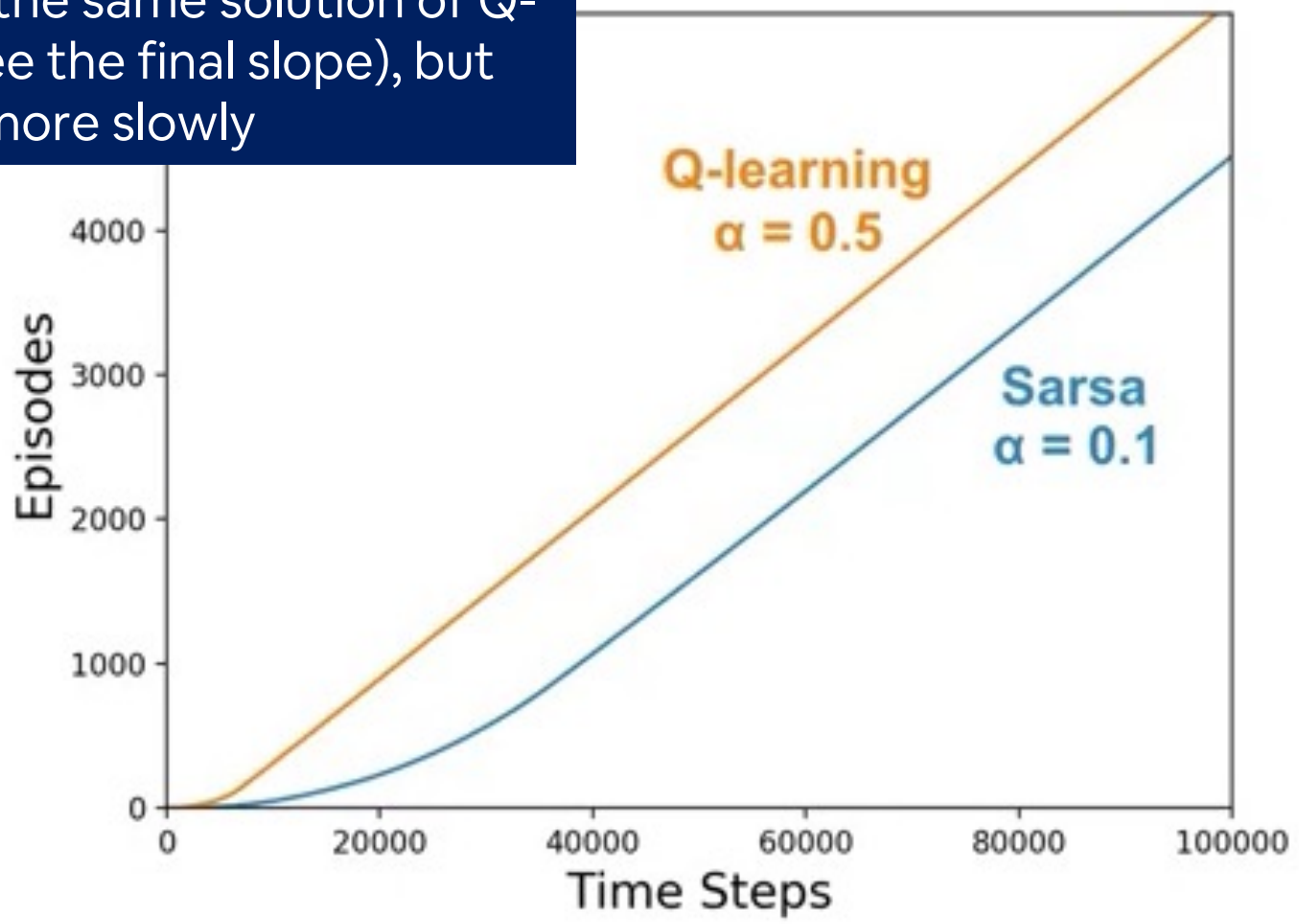


A. White, M. White ‘Sample-based Learning Methods’

Recap - Control: SARSA vs Q-learning – ‘Windy Grid World’ Example #01

SARSA finds the same solution of Q-learning (see the final slope), but more slowly

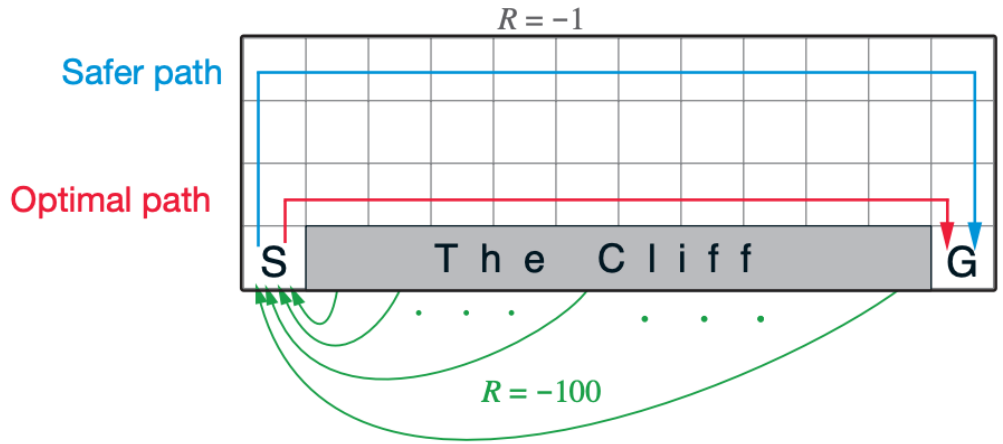




A. White, M. White ‘Sample-based Learning Methods’

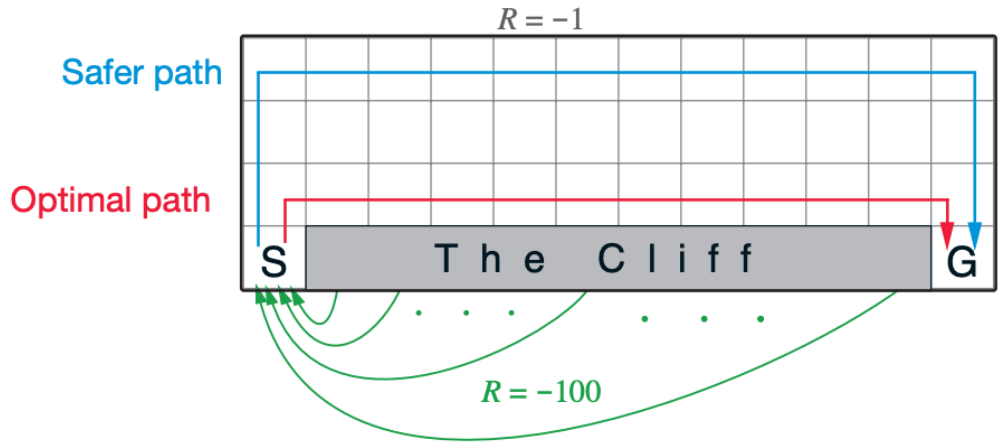
Recap - Control: SARSA vs Q-learning – 'The Cliff Gridworld' Example #02

- Q-learning doesn't iterate between policy evaluation and policy improvement, but rather learns the optimal values directly. Not always ideal!



Recap - Control: SARSA vs Q-learning – 'The Cliff Gridworld' Example #02

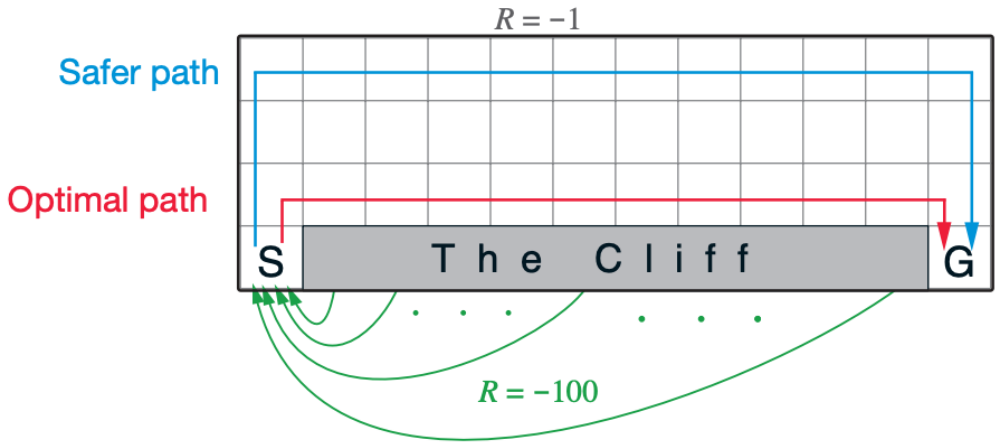
- Q-learning doesn't iterate between policy evaluation and policy improvement, but rather learns the optimal values directly. Not always ideal!
- Since Q-learning learns the optimal value function, it quickly learns that an optimal policy travels right alongside the cliff.
- However, since his actions or epsilon greedy, traveling alongside the cliff occasionally results and falling off the cliff.

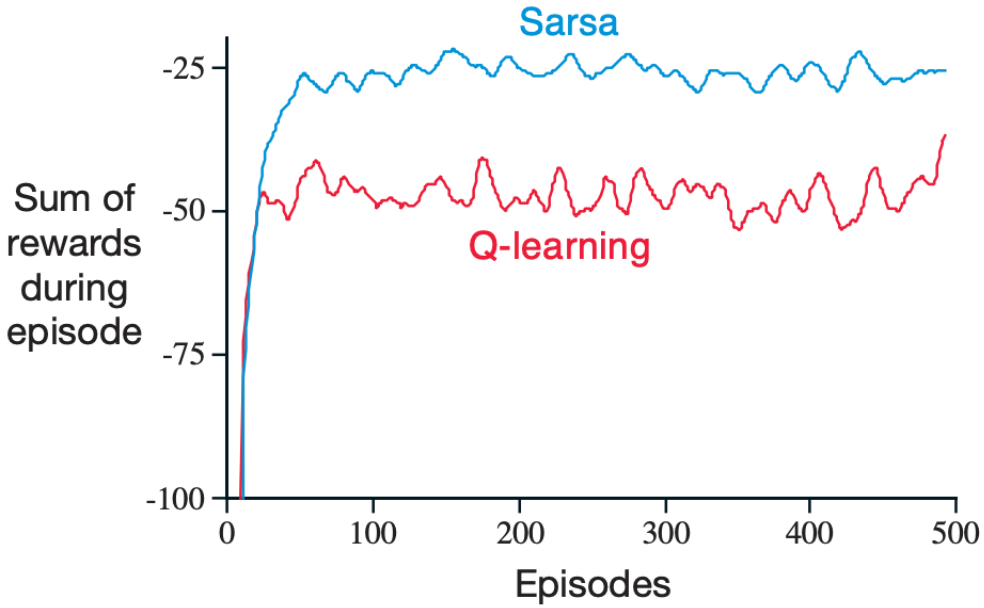


Q-learning trained agent falls down the cliff sometimes!

Recap - Control: SARSA vs Q-learning – 'The Cliff Gridworld' Example #02

- Q-learning doesn't iterate between policy evaluation and policy improvement, but rather learns the optimal values directly. Not always ideal!
- Since Q-learning learns the optimal value function, it quickly learns that an optimal policy travels right alongside the cliff.
- However, since his actions or epsilon greedy, traveling alongside the cliff occasionally results and falling off the cliff.
- Sarsa learns about his current policy, considering the effect of epsilon greedy action selection.





Control: Expected SARSA

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a)\left(r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s',a')\right)$$

$\sim p(s',r|s,a)$ $\pi(a'|s')$

Sarsa: $$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha\left(R_{t+1} + \gamma Q(S_{t+1},A_{t+1}) - Q(S_t,A_t)\right)$$

SARSA is a sample-based version of Policy Iteration that exploits the Bellman Expectation Equation: it approximates the expectation by sampling from the environment and from its policy

A. White, M. White 'Sample-based Learning Methods'

Control: Expected SARSA

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a)\left(r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s',a')\right)$$

$\sim p(s',r|s,a)$ $\sim \pi(a'|s')$

Sarsa: $$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha\left(R_{t+1} + \gamma Q(S_{t+1},A_{t+1}) - Q(S_t,A_t)\right)$$

SARSA is a sample-based version of Policy Iteration that exploits the Bellman Expectation Equation: it approximates the expectation by sampling from the environment and from its policy

However, the policy is already known by the agent: why sampling its next action?

A. White, M. White 'Sample-based Learning Methods'

Control: Expected SARSA

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a)\left(r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s',a')\right)$$

$\sim p(s',r|s,a)$ $\sim \pi(a'|s')$

Sarsa: $$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha\left(R_{t+1} + \gamma Q(S_{t+1},A_{t+1}) - Q(S_t,A_t)\right)$$

$$\sum_{a'} \pi(a'|S_{t+1}) Q(S_{t+1},a')$$

The agent can compute an expectation: the weighted* sum of all possible next actions

*weights are the probability of next action given the current policy

A. White, M. White 'Sample-based Learning Methods'

Control: Expected SARSA

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \Big[R_{t+1} + \gamma \mathbb{E}_\pi [Q(S_{t+1}, A_{t+1}) \mid S_{t+1}] - Q(S_t, A_t) \Big]$$

$$= Q(S_t, A_t) + \alpha \Big[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \Big],$$

Control: Expected SARSA

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \mathbb{E}_\pi[Q(S_{t+1}, A_{t+1}) \mid S_{t+1}] - Q(S_t, A_t)\Big]$$
$$= Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)\Big],$$

At each step, the agent must average the next state's action values according to how likely they are under the policy.

$Q(S_{t+1}, a') =$	0.0	−1.0	2.0	1.0	
$\pi(a' \mid S_{t+1}) =$	0.1	0.1	0.7	0.1	
$\sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a') =$	0.0	− 0.1	+ 1.4	+ 0.1	= 1.4

A. White, M. White 'Sample-based Learning Methods'

Control: Expected SARSA

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \mathbb{E}_\pi[Q(S_{t+1}, A_{t+1}) \mid S_{t+1}] - Q(S_t, A_t)\Big]$$

$$= Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)\Big],$$

Do you think SARSA or Expected SARSA is better?

At each step, the agent must average the next state's action values according to how likely they are under the policy.

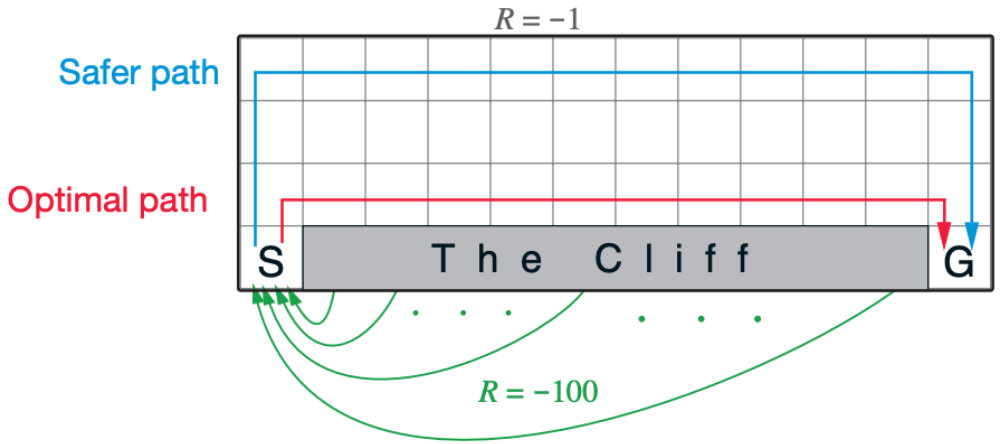
$Q(S_{t+1}, a') =$	0.0	−1.0	2.0	1.0
$\pi(a' \mid S_{t+1}) =$	0.1	0.1	0.7	0.1

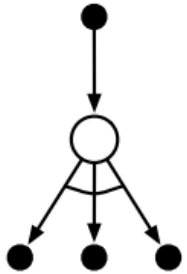
$$\sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a') = 0.0 - 0.1 + 1.4 + 0.1 = 1.4$$

A. White, M. White 'Sample-based Learning Methods'

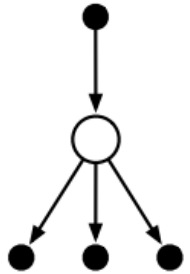
Control: Expected SARSA vs. SARSA

- Expected SARSA has slower updates, but it is more stable (lower variance than SARSA).





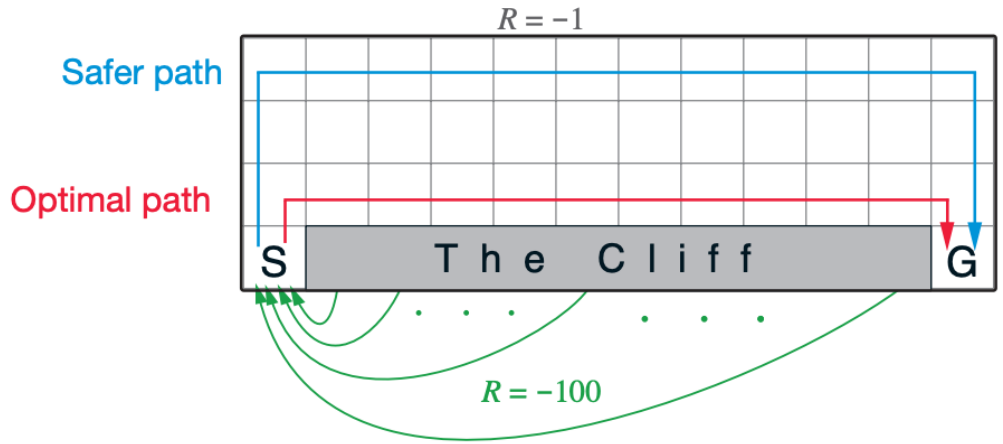
Q-learning

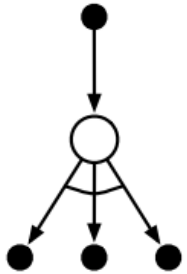


Expected Sarsa

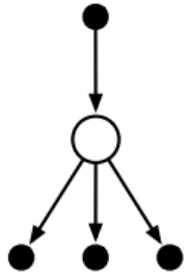
Control: Expected SARSA vs. SARSA

- Expected SARSA has slower updates, but it is more stable (lower variance than SARSA).
- Expected SARSA moves deterministically in the same direction as SARSA moves in expectation.





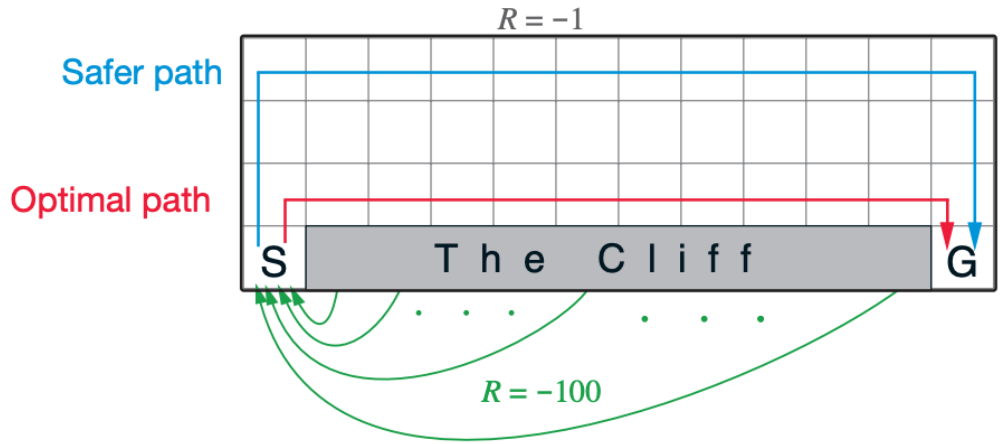
Q-learning

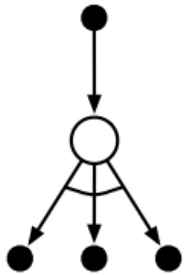


Expected Sarsa

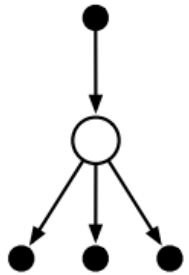
Control: Expected SARSA vs. SARSA

- Expected SARSA has slower updates, but it is more stable (lower variance than SARSA).
- Expected SARSA moves deterministically in the same direction as SARSA moves in expectation.
- Thanks to its more stable updates, Expected SARSA can handle higher values of α

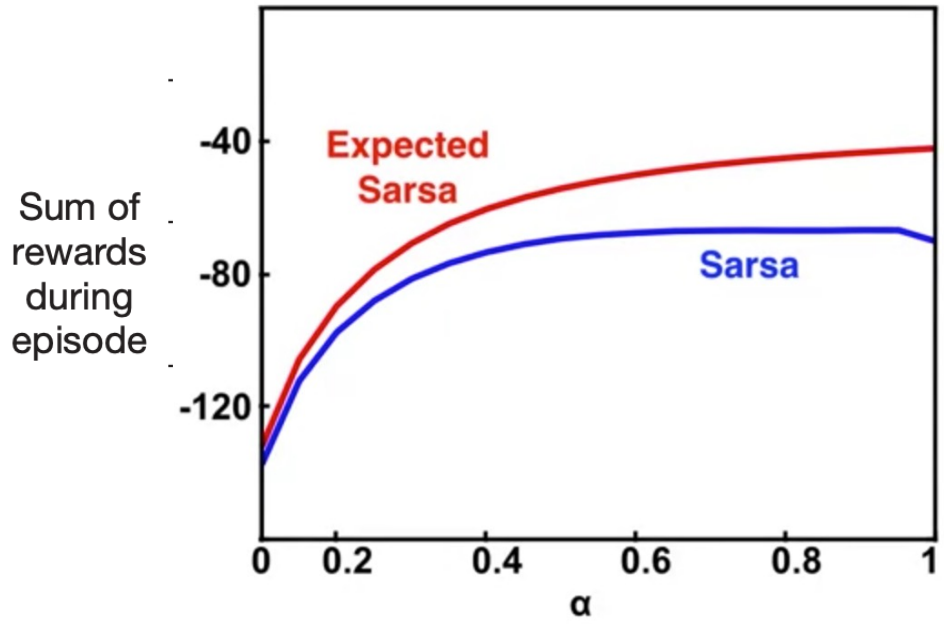




Q-learning



Expected Sarsa



Control: TD control and Bellman equations

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a)\left(r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s',a')\right)$$

$$q_*(s,a) = \sum_{s',r} p(s',r|s,a)\left(r + \gamma \max_{a'} q_*(s',a')\right)$$

Sarsa

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

Expected Sarsa

$$R_{t+1} + \gamma \sum_{a'} \pi(a'|S_{t+1}) Q(S_{t+1}, a')$$

Q-learning

$$R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$$

A. White, M. White 'Sample-based Learning Methods'

Control: TD control and Bellman equations

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a)\left(r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s',a')\right)$$

$$q_*(s,a) = \sum_{s',r} p(s',r|s,a)\left(r + \gamma \max_{a'} q_*(s',a')\right)$$

Sarsa

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

On-policy

Expected Sarsa

$$R_{t+1} + \gamma \sum_{a'} \pi(a'|S_{t+1}) Q(S_{t+1}, a')$$

?

Q-learning

$$R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$$

Off-policy

A. White, M. White 'Sample-based Learning Methods'

Control: TD control and Bellman equations

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a)\left(r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s',a')\right)$$

$$q_*(s,a) = \sum_{s',r} p(s',r|s,a)\left(r + \gamma \max_{a'} q_*(s',a')\right)$$

Sarsa

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

On-policy

Expected Sarsa

$$R_{t+1} + \gamma \sum_{a'} \pi(a'|S_{t+1}) Q(S_{t+1}, a')$$

Off-policy

Q-learning

$$R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$$

Off-policy

You look ahead and average over potential next actions and then you update Q(s,a)!

A. White, M. White 'Sample-based Learning Methods'

Algorithm 15: Expected Sarsa

Input: policy π, positive integer *num_episodes*, small positive fraction α
Output: value function Q ($\approx q_\pi$ if *num_episodes* is large enough)
Initialize Q arbitrarily (e.g., $Q(s,a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(\textit{terminal-state}, \cdot) = 0$)
for $i \leftarrow 1$ ***to*** *num_episodes* **do**

Observe S_0
$t \leftarrow 0$
repeat
Choose action A_t using policy derived from Q (e.g., ϵ-greedy)
Take action A_t and observe R_{t+1}, S_{t+1}
$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t))$
$t \leftarrow t + 1$
until S_t *is terminal*;

end
return Q

You look ahead and average over potential next actions and then you update Q(s,a)

TD-learning methods: Exam

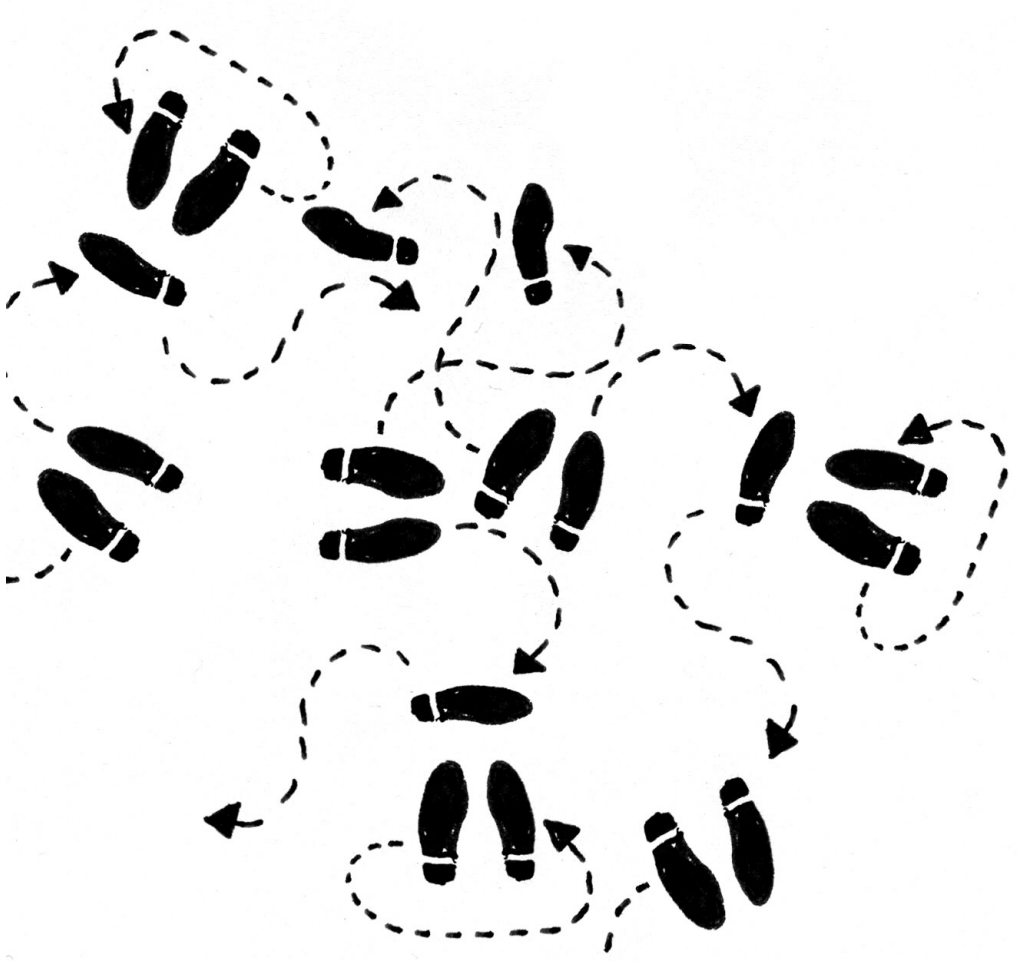
- All the content of Chapter 6 are Exam material, beside:

i. Section 6.8 can be considered optional

ii. Sections 6.7 can be skipped

- (Again!) Pay particular attention to the algorithms!

n-step Bootstrapping (Chapter 7)



MC vs. TD(0)

MC: no bias

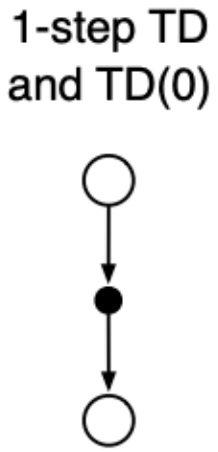
TD(0): low variance

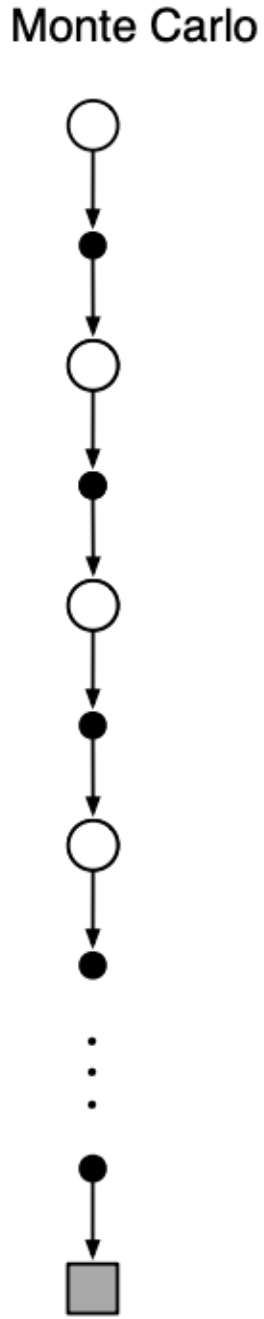
MC uses real return

TD(0) just rewards and bootstrap

MC updates just at the end of the episode

TD(0) updates along the way





Prediction: MC vs. TD(0)

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots$$

MC: $$V(S_t) \leftarrow V(S_t) + \alpha\,(G_t - V(S_t))$$

TD(0): $$V(S_t) \leftarrow V(S_t) + \alpha\,(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

The TD target can be seen as the 'best' estimation of G_t at time t+1:

$$G_t = R_{t+1} + \gamma G(S_{t+1}) \sim R_{t+1} + \gamma V(S_{t+1})$$

Prediction: why not considering a different estimation of G, for example after 2 steps?

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots$$

$n = 1$ $\quad G_{t:t+1} \doteq R_{t+1} + \gamma V_t(S_{t+1})$

$n = 2$ $\quad G_{t:t+2} \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+1}(S_{t+2})$

Prediction: why not considering a different estimation of G, for example after 2 steps?

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots$$

This is the best estimation of *G* at time *t+2*!

$n = 1$ $\quad G_{t:t+1} \doteq R_{t+1} + \gamma V_t(S_{t+1})$

$n = 2$ $\quad G_{t:t+2} \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+1}(S_{t+2})$

Prediction: why not considering a different estimation of G, for example after 2 steps?

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots$$

This is the best estimation of *G* at time *t+2*!

$n = 1$ $\quad G_{t:t+1} \doteq R_{t+1} + \gamma V_t(S_{t+1})$

$n = 2$ $\quad G_{t:t+2} \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+1}(S_{t+2})$

I can use this quantity as a different *TD target*

Prediction: why not considering a different estimation of G, for example after 2 steps?

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots$$

This is the best estimation of *G* at time *t+2*!

$n = 1$ $\quad G_{t:t+1} \doteq R_{t+1} + \gamma V_t(S_{t+1})$

$n = 2$ $\quad G_{t:t+2} \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+1}(S_{t+2})$

Pay attention to the notation!

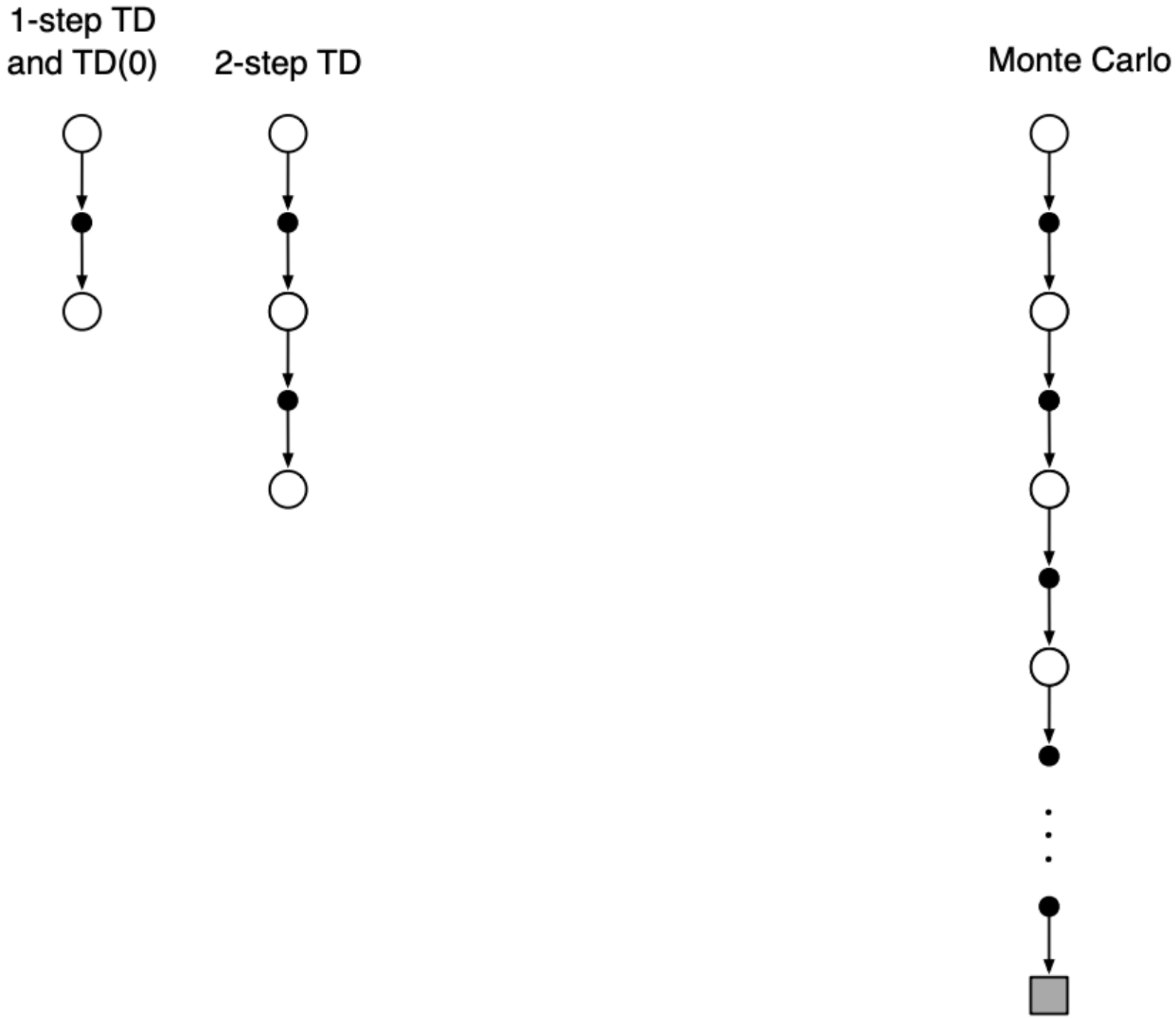
I can use this quantity as a different *TD target*

MC vs. TD(0)

MC: low bias

TD(0): low variance

Why don’t using a compromise between MC and TD(0)?

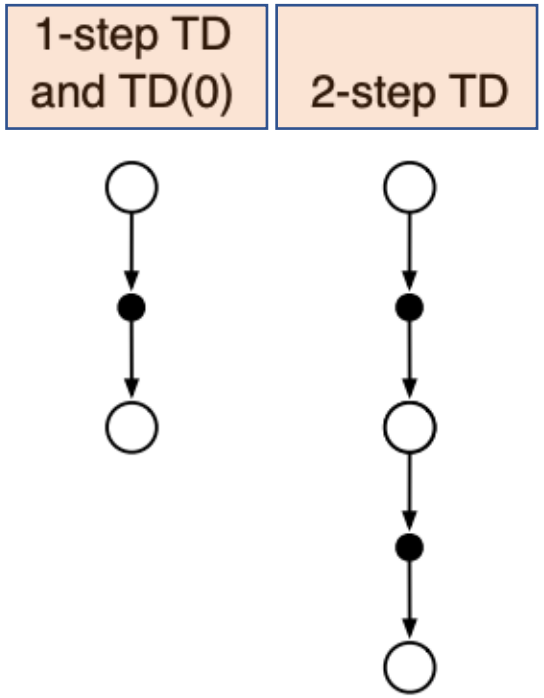


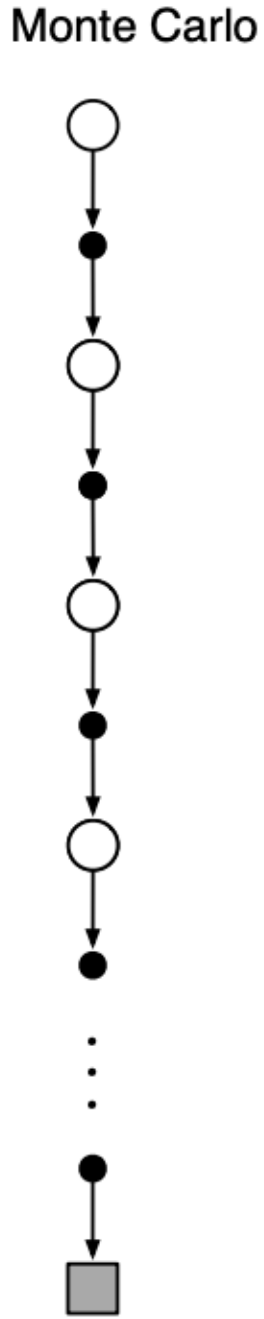
MC vs. TD(0)

MC: low bias

TD(0): low variance

Why don’t using a compromise between MC and TD(0)?





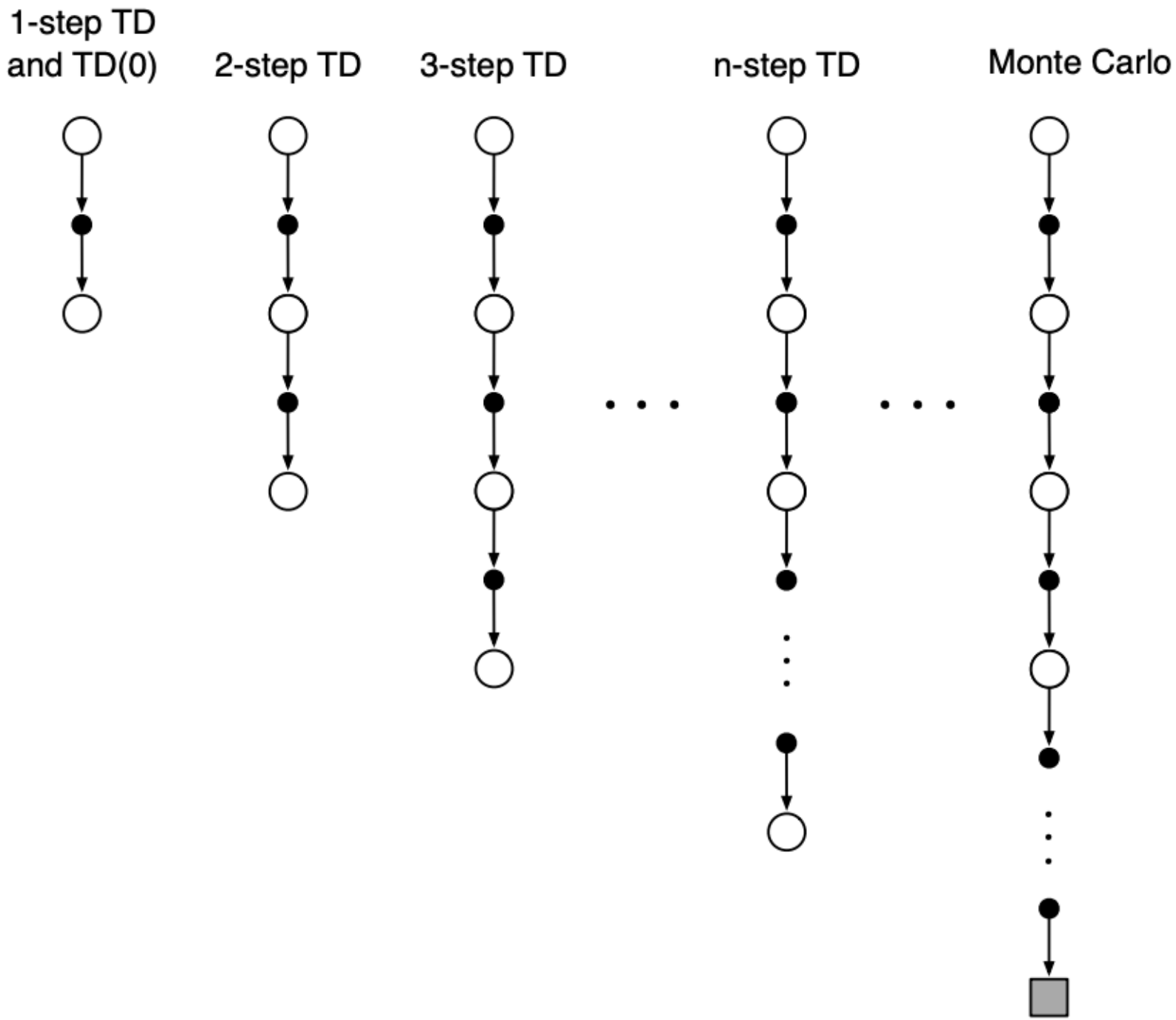
MC vs. TD(0)

MC: low bias

TD(0): low variance

Why don't using a compromise between MC and TD(0)?

We can generalize by considering n-step TD!



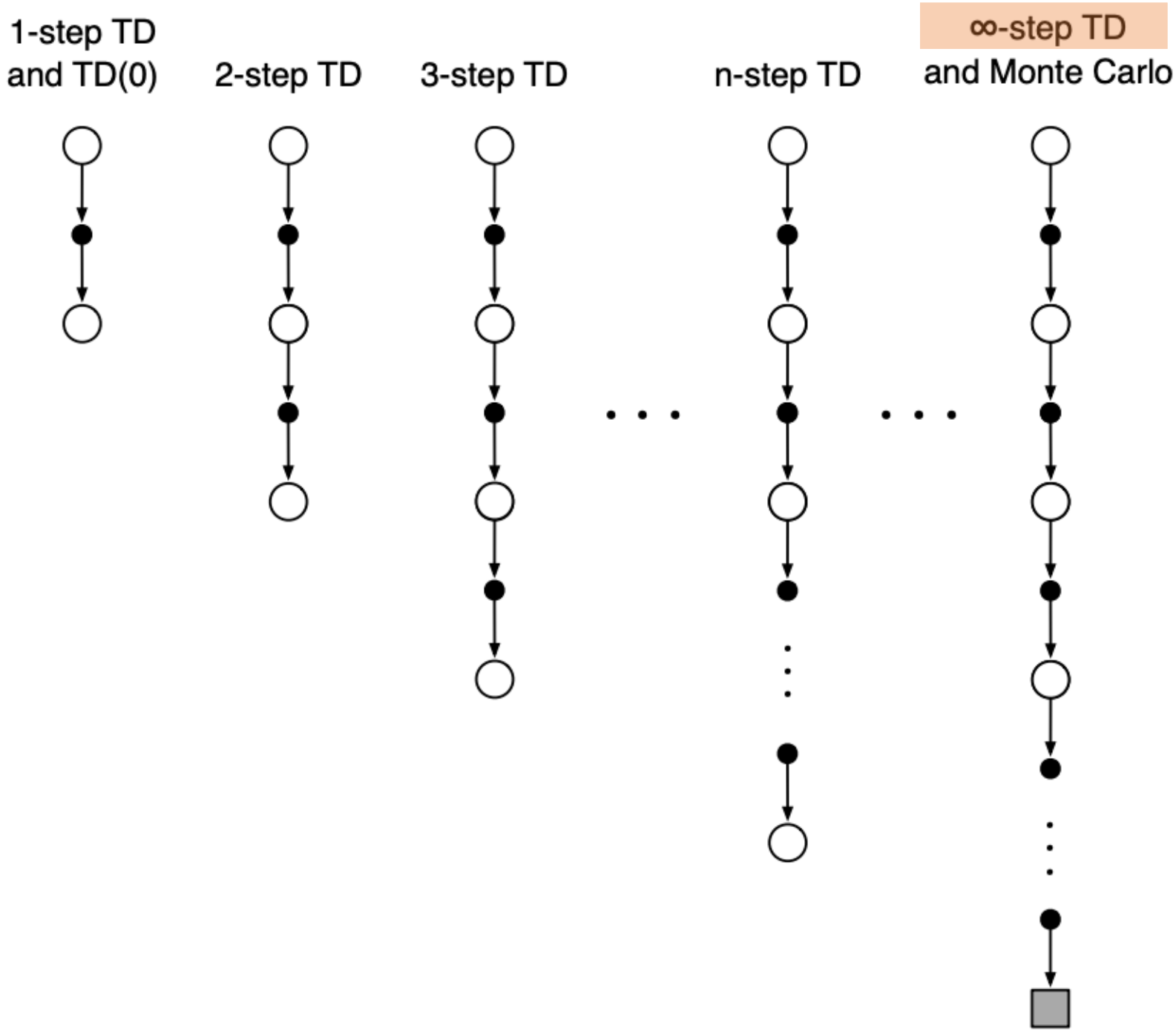
MC vs. TD(0)

MC: low bias

TD(0): low variance

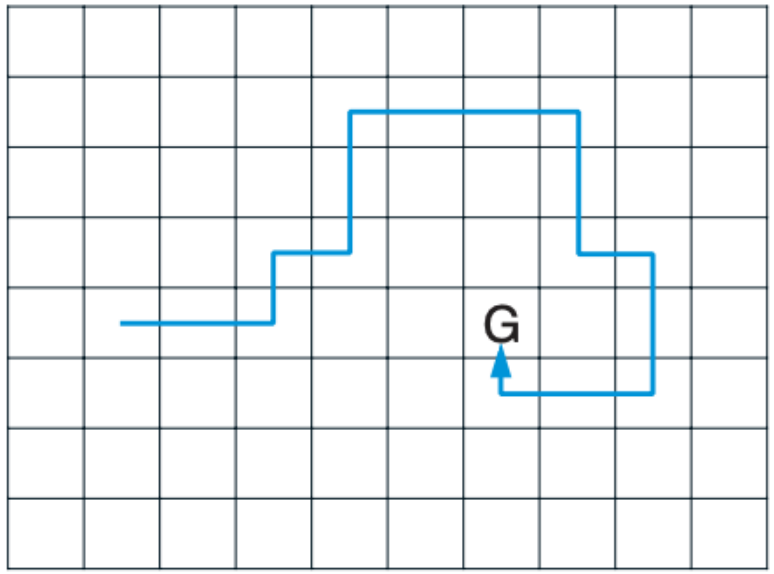
Why don't using a compromise between MC and TD(0)?

We can generalize by considering n-step TD!

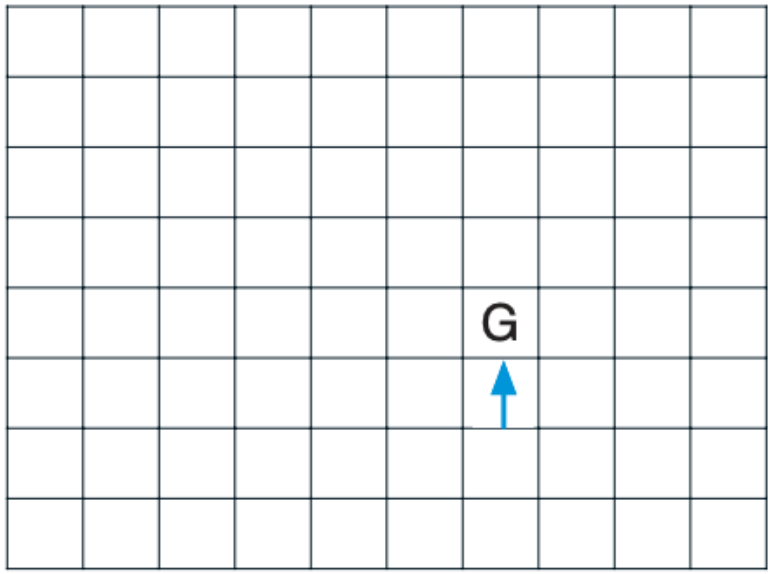


n-step TD learning (Prediction or Control)

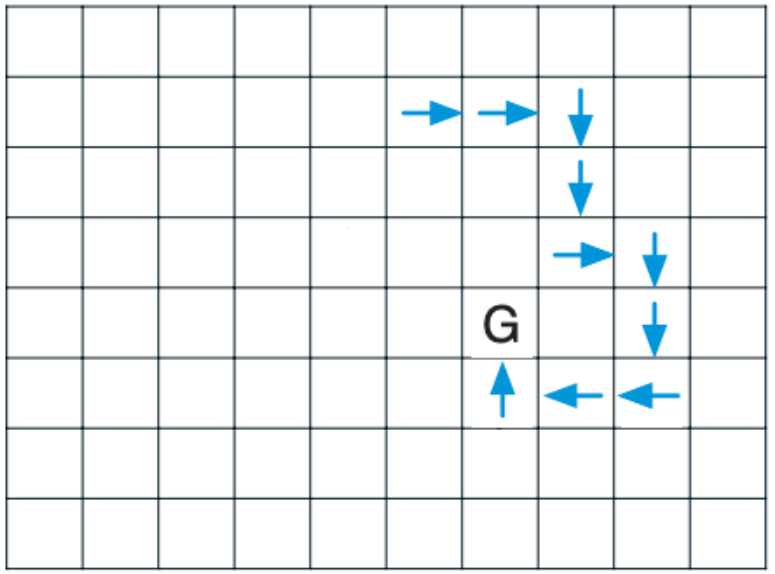
Path taken



Action values increased by one-step Sarsa



Action values increased by 10-step Sarsa



- The one-step method strengthens only the last action of the sequence of actions that led to the high reward
- The n-step method strengthens the last n actions of the sequence, so that much more is learned from the one episode.

Prediction: *n*-step return

Let's define the n-step returns:

$$
\begin{aligned}
n = 1 \quad (TD) \quad & G_{t:t+1} \doteq R_{t+1} + \gamma V_t(S_{t+1}) \\
n = 2 \quad & G_{t:t+2} \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+1}(S_{t+2}) \\
\vdots \quad & \vdots \\
n = \infty \quad (MC) \quad & G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T
\end{aligned}
$$

$$
G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n})
$$

A terminal state

Prediction: *n*-step return

Let's define the n-step returns:

$$
\begin{aligned}
n = 1 \quad (TD) \quad & G_{t:t+1} \doteq R_{t+1} + \gamma V_t(S_{t+1}) \\
n = 2 \quad & G_{t:t+2} \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+1}(S_{t+2}) \\
\vdots \quad & \vdots \\
n = \infty \quad (MC) \quad & G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T
\end{aligned}
$$

$$
G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n})
$$

n-step TD learning will consider the following updates:

$$
V_{t+n}(S_t) \doteq V_{t+n-1}(S_t) + \alpha \left[G_{t:t+n} - V_{t+n-1}(S_t) \right]
$$

Prediction: *n*-step return

Where this is not defined?

Let's define the n-step returns:

$$
\begin{aligned}
n = 1 \quad (TD) \quad & G_{t:t+1} \doteq R_{t+1} + \gamma V_t(S_{t+1}) \\
n = 2 \quad & G_{t:t+2} \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+1}(S_{t+2}) \\
\vdots \quad & \vdots \\
n = \infty \quad (MC) \quad & G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T
\end{aligned}
$$

$$
G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n})
$$

n-step TD learning will consider the following updates:

$$
V_{t+n}(S_t) \doteq V_{t+n-1}(S_t) + \alpha \left[G_{t:t+n} - V_{t+n-1}(S_t) \right]
$$

Prediction: *n*-step return

Where this is not defined? When there are less then *n* step before the terminal state!

Let's define the n-step returns:

$$
\begin{aligned}
n = 1 \quad (TD) \quad & G_{t:t+1} \doteq R_{t+1} + \gamma V_t(S_{t+1}) \\
n = 2 \quad & G_{t:t+2} \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+1}(S_{t+2}) \\
\vdots \quad & \vdots \\
n = \infty \quad (MC) \quad & G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T
\end{aligned}
$$

$$
G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n})
$$

n-step TD learning will consider the following updates:

$$
V_{t+n}(S_t) \doteq V_{t+n-1}(S_t) + \alpha \left[G_{t:t+n} - V_{t+n-1}(S_t) \right]
$$

n-step TD for estimating $V \approx v_\pi$

PREDICTION TASK

Input: a policy π
Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n
Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$
All store and access operations (for S_t and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$, then:
  | Take an action according to $\pi(\cdot|S_t)$
  | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  | If S_{t+1} is terminal, then $T \leftarrow t+1$
  | $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
  | If $\tau \geq 0$:
  | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
  | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ $(G_{\tau:\tau+n})$
  | $V(S_\tau) \leftarrow V(S_\tau) + \alpha\,[G - V(S_\tau)]$
  Until $\tau = T - 1$

A fixed value for n

n-step TD for estimating $V \approx v_\pi$

t will be our step counter inside the episode

Input: a policy π
Algorithm parameters: step size $\alpha \in (0, 1]$, a positiv
Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$
All store and access operations (for S_t and R_t) can

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$, then:
  | Take an action according to $\pi(\cdot|S_t)$
  | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  | If S_{t+1} is terminal, then $T \leftarrow t + 1$
  | $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
  | If $\tau \geq 0$:
  | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
  | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ $(G_{\tau:\tau+n})$
  | $V(S_\tau) \leftarrow V(S_\tau) + \alpha\,[G - V(S_\tau)]$
  Until $\tau = T - 1$

n-step TD for estimating $V \approx v_\pi$

Input: a policy π
Algorithm parameters: step size $\alpha \in (0, 1]$, a positiv
Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$
All store and access operations (for S_t and R_t) can

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
    If $t < T$, then:
      Take an action according to $\pi(\cdot|S_t)$
      Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
      If S_{t+1} is terminal, then $T \leftarrow t + 1$
    $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
    If $\tau \geq 0$:
      $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
      If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ $(G_{\tau:\tau+n})$
      $V(S_\tau) \leftarrow V(S_\tau) + \alpha [G - V(S_\tau)]$
  Until $\tau = T - 1$

t will be our step counter inside the episode
We will use:
- τ for starting the updates only when n steps are actually seen in the trajectory
- T for checking if we have arrived to close to a terminal state (and therefore a different update must be given)

We need to have at least n steps to do our updates: if we are too close to a terminal state we can't update anymore

n-step TD for estimating $V \approx v_\pi$

Input: a policy π
Algorithm parameters: step size $\alpha \in (0, 1]$, a positiv
Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$
All store and access operations (for S_t and R_t) can

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
    If $t < T$, then:
      Take an action according to $\pi(\cdot|S_t)$
      Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
      If S_{t+1} is terminal, then $T \leftarrow t + 1$
    $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
    If $\tau \geq 0$:
      $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
      If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ $(G_{\tau:\tau+n})$
      $V(S_\tau) \leftarrow V(S_\tau) + \alpha [G - V(S_\tau)]$
  Until $\tau = T - 1$

t will be our step counter inside the episode
We will use:
- τ for starting the updates only when n steps are actually seen in the trajectory
- T for checking if we have arrived to close to a terminal state (and therefore a different update must be given)

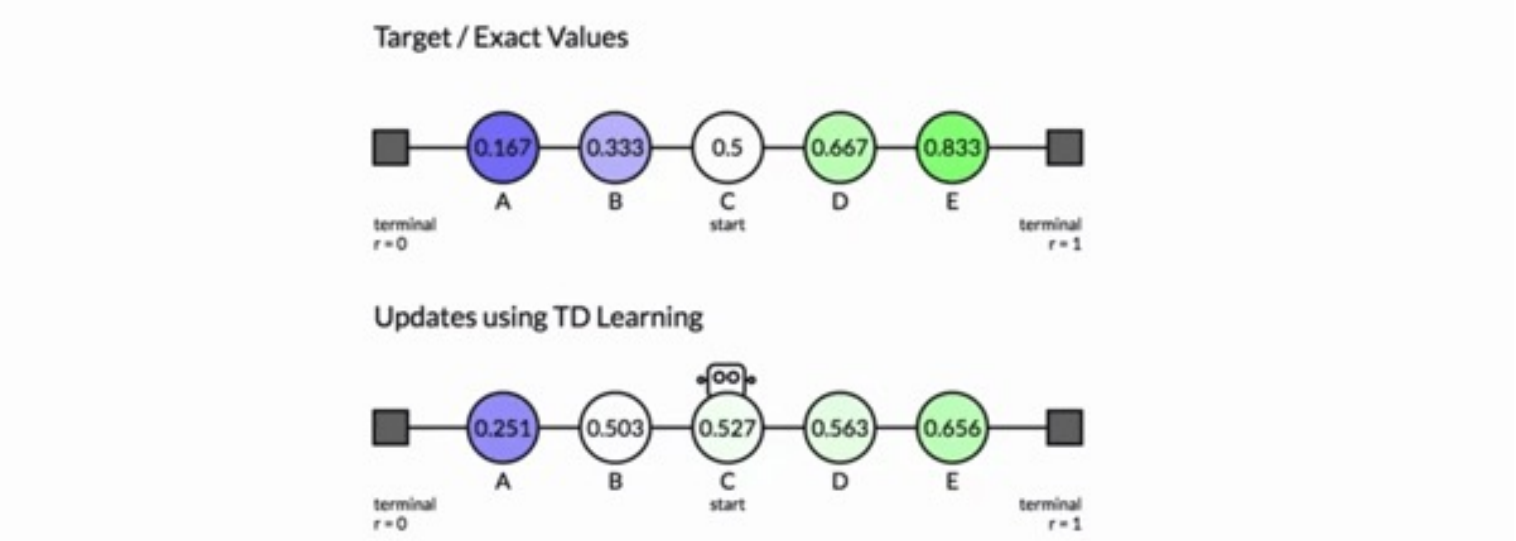
We need to have at least n steps to do our updates: if we are too close to a terminal state we can't update anymore

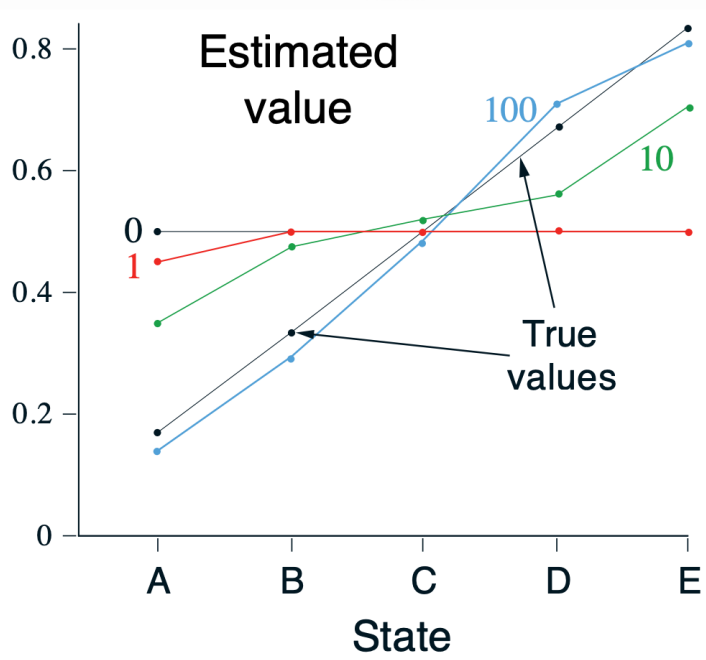
Policy-evaluation

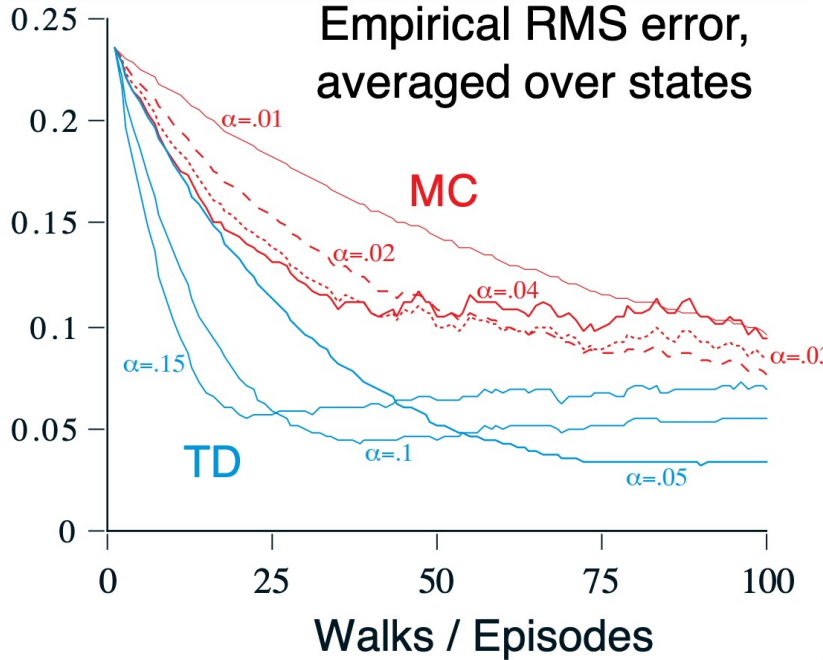
Prediction: *n*-step return – example

What is the ‘best’ n? It depends!

Let’s consider the random walk process example (we are evaluating a random uniform policy): instead of 5 states, we consider 19 states!



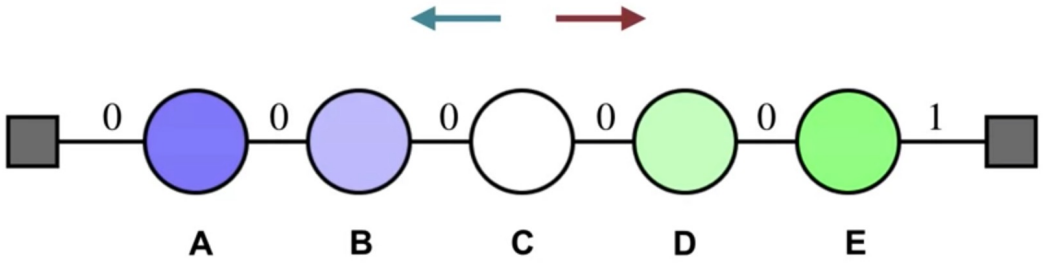




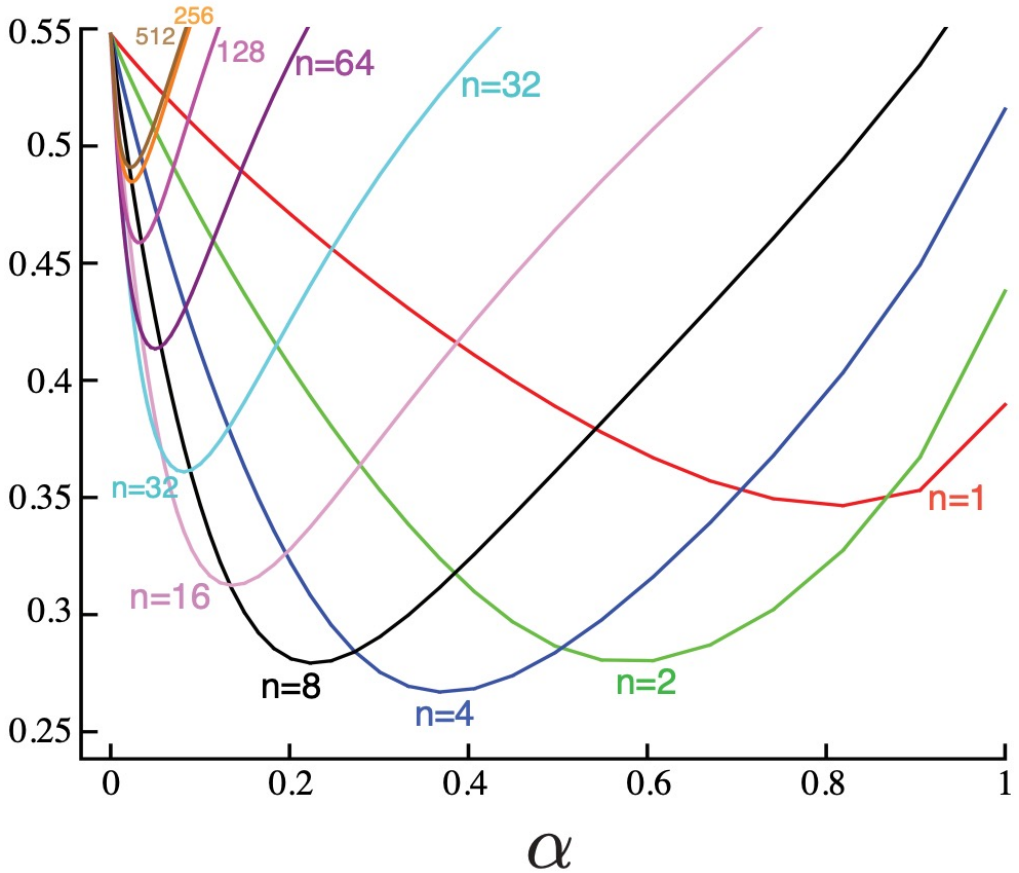
Prediction: *n*-step return – example

What is the ‘best’ n? It depends!

Let’s consider the random walk process example (we are evaluating a random uniform policy): instead of 5 states, we consider 19 states!



Average RMS error over 19 states and first 10 episodes

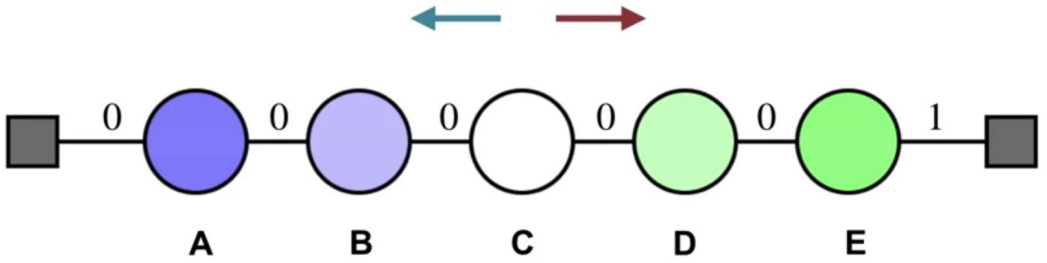


Prediction: *n*-step return – example

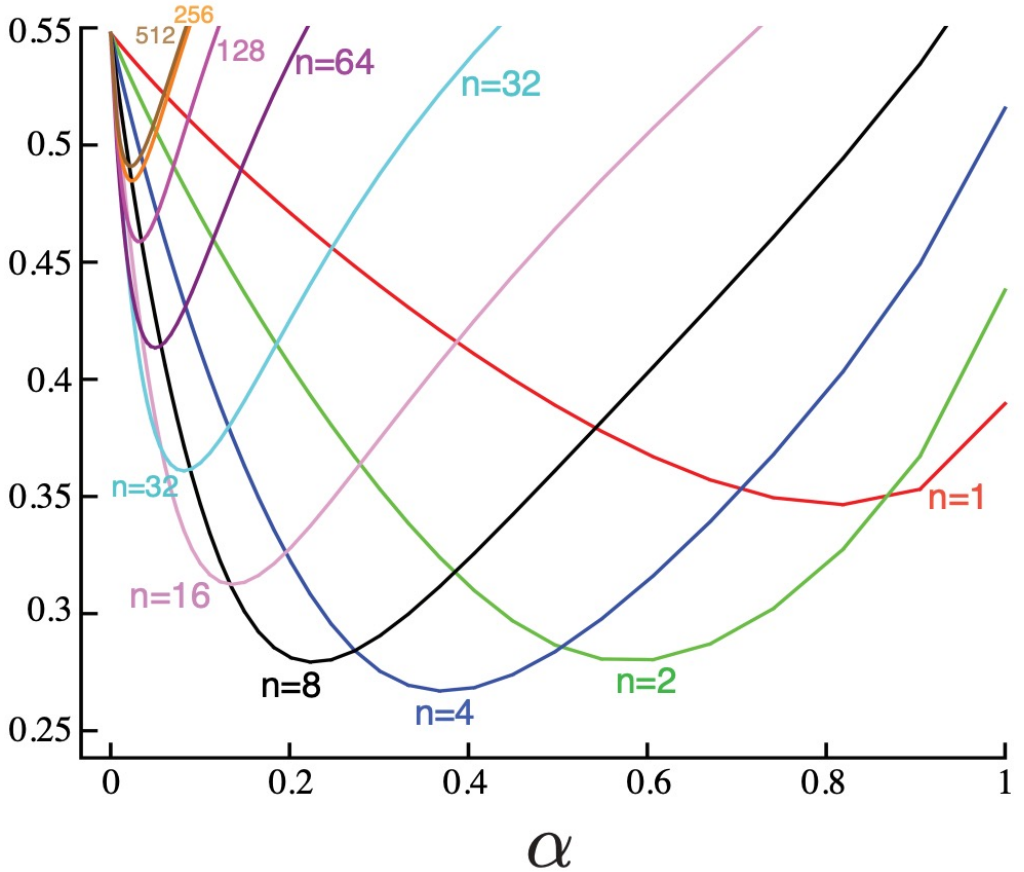
What is the ‘best’ n? It depends!

Let’s consider the random walk process example (we are evaluating a random uniform policy): instead of 5 states, we consider 19 states!

With TD-lambda we will be able to ‘exploit’ all possible values of n (Chapter 12 – eligibility traces)



Average RMS error over 19 states and first 10 episodes



Control: *n*-step SARSA

Let's define the n-step returns:

$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^{n} Q_{t+n-1}(S_{t+n}, A_{t+n})$$

n-step SARSA updates:

$$Q_{t+n}(S_t, A_t) \doteq Q_{t+n-1}(S_t, A_t) + \alpha \left[G_{t:t+n} - Q_{t+n-1}(S_t, A_t) \right]$$

n-step Sarsa for estimating $Q \approx q_*$ or q_π

CONTROL TASK

Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be ε-greedy with respect to Q, or to a fixed given policy
Algorithm parameters: step size $\alpha \in (0,1]$, small $\varepsilon > 0$, a positive integer n
All store and access operations (for S_t, A_t, and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  Select and store an action $A_0 \sim \pi(\cdot|S_0)$
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$, then:
  | Take action A_t
  | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  | If S_{t+1} is terminal, then:
  | $T \leftarrow t+1$
  | else:
  | Select and store an action $A_{t+1} \sim \pi(\cdot|S_{t+1})$
  | $\tau \leftarrow t-n+1$ (τ is the time whose estimate is being updated)
  | If $\tau \geq 0$:
  | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
  | If $\tau+n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ $(G_{\tau:\tau+n})$
  | $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\left[G - Q(S_\tau, A_\tau)\right]$
  | If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is ε-greedy wrt Q
  Until $\tau = T-1$

A fixed value for n

Loop for each episode:

Initialize and store $S_0 \neq$ terminal

Select and store an action $A_0 \sim \pi(\cdot|S_0)$

$T \leftarrow \infty$

Loop for $t = 0, 1, 2, \ldots$:

If $t < T$, then:

Take action A_t

Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

If S_{t+1} is terminal, then:

$T \leftarrow t + 1$

else:

Select and store an action $A_{t+1} \sim \pi(\cdot|S_{t+1})$

$\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)

If $\tau \geq 0$:

$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$

If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ $(G_{\tau:\tau+n})$

$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\,[G - Q(S_\tau, A_\tau)]$

If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is ε-greedy wrt Q

Until $\tau = T - 1$

t will be our step counter inside the episode

We will use:

- τ for starting the updates only when n steps are actually seen in the trajectory
- T for checking if we have arrived to close to a terminal state (and therefore a different update must be given)

Loop for each episode:

Initialize and store $S_0 \neq$ terminal

Select and store an action $A_0 \sim \pi(\cdot|S_0)$

$T \leftarrow \infty$

Loop for $t = 0, 1, 2, \ldots$:

| If $t < T$, then:

| | Take action A_t

| | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

| | If S_{t+1} is terminal, then:

| | | $T \leftarrow t + 1$

| | else:

| | | Select and store an action $A_{t+1} \sim \pi(\cdot|S_{t+1})$

| $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)

| If $\tau \geq 0$:

| | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$

| | If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ $(G_{\tau:\tau+n})$

| | $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\,[G - Q(S_\tau, A_\tau)]$

| | If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is ε-greedy wrt Q

Until $\tau = T - 1$

Policy-evaluation

Policy-improvement

EXPECTED

n-step Sarsa for estimating $Q \approx q_*$ or q_π

CONTROL TASK

Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be ε-greedy with respect to Q, or to a fixed given policy
Algorithm parameters: step size $\alpha \in (0,1]$, small $\varepsilon > 0$, a positive integer n
All store and access operations (for S_t, A_t, and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  Select and store an action $A_0 \sim \pi(\cdot|S_0)$
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
    If $t < T$, then:
      Take action A_t
      Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
      If S_{t+1} is terminal, then:
        $T \leftarrow t+1$
      else:
        Select and store an action $A_{t+1} \sim \pi(\cdot|S_{t+1})$
    $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
    If $\tau \geq 0$:
      $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
      If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)
      $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\,[G - Q(S_\tau, A_\tau)]$
      If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is ε-greedy wrt Q
  Until $\tau = T - 1$

We only have to change one line of code:

$$G_{t:t+n} \doteq R_{t+1} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \bar{V}_{t+n-1}(S_{t+n})$$

$$\bar{V}_t(s) \doteq \sum_a \pi(a|s) Q_t(s,a)$$

Control: off-policy *n*-step SARSA

We can also have off-policy *n*-step, SARSA. We need to consider:

$$V_{t+n}(S_t) \doteq V_{t+n-1}(S_t) + \alpha \rho_{t:t+n-1} \left[G_{t:t+n} - V_{t+n-1}(S_t) \right]$$

where an importance sampling ratio is in place

$$\rho_{t:h} \doteq \prod_{k=t}^{\min(h, T-1)} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$$

Off-policy n-step Sarsa for estimating $Q \approx q_*$ or q_π

Input: an arbitrary behavior policy b such that $b(a|s) > 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be greedy with respect to Q, or as a fixed given policy
Algorithm parameters: step size $\alpha \in (0,1]$, a positive integer n
All store and access operations (for S_t, A_t, and R_t) can take their index mod $n+1$

Loop for each episode:
    Initialize and store $S_0 \neq$ terminal
    Select and store an action $A_0 \sim b(\cdot|S_0)$
    $T \leftarrow \infty$
    Loop for $t = 0, 1, 2, \ldots$:
    | If $t < T$, then:
    | | Take action A_t
    | | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
    | | If S_{t+1} is terminal, then:
    | | | $T \leftarrow t+1$
    | | else:
    | | | Select and store an action $A_{t+1} \sim b(\cdot|S_{t+1})$
    | $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
    | If $\tau \geq 0$:
    | | $\rho \leftarrow \prod_{i=\tau+1}^{\min(\tau+n, T-1)} \frac{\pi(A_i|S_i)}{b(A_i|S_i)}$ ($\rho_{\tau+1:\tau+n}$)
    | | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
    | | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)
    | | $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\rho\left[G - Q(S_\tau, A_\tau)\right]$
    | | If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is greedy wrt Q
    Until $\tau = T - 1$

Loop for each episode:

Initialize and store $S_0 \neq$ terminal

Select and store an action $A_0 \sim b(\cdot|S_0)$

Actions are taken according to a behaviour policy

$T \leftarrow \infty$

Loop for $t = 0, 1, 2, \ldots$:

| If $t < T$, then:

| | Take action A_t

| | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

| | If S_{t+1} is terminal, then:

| | | $T \leftarrow t + 1$

| | else:

| | | Select and store an action $A_{t+1} \sim b(\cdot|S_{t+1})$

| $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)

| If $\tau \geq 0$:

| | $\rho \leftarrow \prod_{i=\tau+1}^{\min(\tau+n, T-1)} \frac{\pi(A_i|S_i)}{b(A_i|S_i)}$ $(\rho_{\tau+1:\tau+n})$

| | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

Target policy

| | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ $(G_{\tau:\tau+n})$

| | $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\rho\,[G - Q(S_\tau, A_\tau)]$

| | If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is greedy wrt Q

Until $\tau = T - 1$

Loop for each episode:

Initialize and store $S_0 \neq$ terminal

Select and store an action $A_0 \sim b(\cdot|S_0)$

$T \leftarrow \infty$

Loop for $t = 0, 1, 2, \ldots$:

| If $t < T$, then:

| | Take action A_t

| | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

| | If S_{t+1} is terminal, then:

| | | $T \leftarrow t + 1$

| | else:

| | | Select and store an action $A_{t+1} \sim b(\cdot|S_{t+1})$

| $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)

| If $\tau \geq 0$:

| | $\rho \leftarrow \prod_{i=\tau+1}^{\min(\tau+n, T-1)} \frac{\pi(A_i|S_i)}{b(A_i|S_i)}$ ($\rho_{\tau+1:\tau+n}$)

| | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

| | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)

| | $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\rho\,[G - Q(S_\tau, A_\tau)]$

| | If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is greedy wrt Q

Until $\tau = T - 1$

Importance Sampling must be considered

Target policy

n-step bootstrap: Exam

- Only Section 7.1, 7.2 and 7.3 are exam material
- (Again!) Pay particular attention to the algorithms!

What to expect

- How to scale up: approximate solution methods! Supervised learning at service of Reinforcement Learning (Part II of the book)
- In practice, the supervised learning approaches in RL are linear approaches or Deep Learning approaches.
- Policy learning approaches
- Applications

Credits

- Image of the course is taken from C. Mahoney ‘Reinforcement Learning’ https://towardsdatascience.com/reinforcement-learning-fda8ff535bb6

Università degli Studi di Padova

Reinforcement Learning 2024/2025

RL2 Reinforcement Learning Research Lab @ UniPD

Thank you! Questions?

Gian Antonio Susto

Ruggero Carli